Principal Component Analysis

Foundations of Machine Learning CS 463/663

ANNOUNCEMENTS

Announcements

- **Reminder Project 1:** due Monday (2/16)

Basic ML Algorithms Plan & Overview

WHAT IS ML

	Unsupervised	Supervised
Continuous	**Analysis**: You do not know the target. It does not require or predict a target variable and uses continuous inputs. 1 • Principal Component Analysis	**Regression**: You know the target. Target is a real number. 3 • Linear, LASSO, Ridge, Polynomial • K-Nearest Neighbours • Decision Trees (CART) & Ensembles • Deep Neural Networks - regression
Discrete	**Clustering**: You do not know the target. Target is a class / category / grouping. 2 • k-Means • DBSCAN, Hierarchical	**Classification**: You know the target. Target is a class / category / grouping. 4 • Logistic Regression • K-Nearest Neighbours • Decision Trees (CART) & Ensembles • Deep Neural Networks - classification

AGENDA

01

MOTIVATION FOR PCA

02

STEPS OF PCA

03

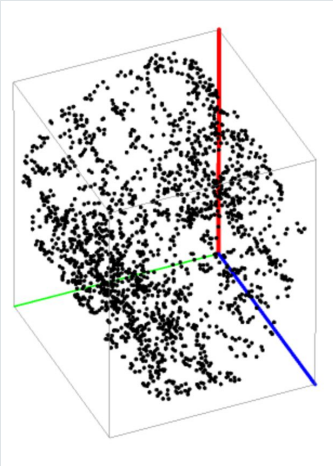
MATH OVERVIEW OF PCA

01 MOTIVATION FOR PCA

MOTIVATION FOR PCA

Intuition behind PCA

- How can we reduce the number of dimensions (i.e., features) of our dataset without losing important information?
- https://demonstrations.wolfram.com/ProjectingImagesFromAPointCloud/



MOTIVATION FOR PCA

Why PCA?

- Why do we want to do this?
  - Overcome the *curse of dimensionality*, where high dimensional data leads to:
    - Computational inefficiency
    - Redundant or highly correlated features
    - Overfitting in machine learning models
      - Model learns patterns specific to a particular subset of data (i.e., what it is trained on) and is not generalizable to other datasets
      - Performance on training → great
      - Performance on test and other data → poor

MOTIVATION FOR PCA

Why PCA?

- Why do we want to do this?
  - Difficult to plot and interpret high-dimensional data
    - We plotted one variable against another but with more than pairwise comparisons this falls apart quickly
  - Noise reduction
    - PCA helps reduce the noise in the data by focusing only on the most important features
      - Think of noise as irrelevant or random data that has an influence but isn't meaningful (e.g., fitness tracker app where one tracker wasn't working properly out of 40 or one person that forgot to wear their tracker for a few days)

MOTIVATION FOR PCA

Goal of PCA

- Ultimately, we want the **minimum amount of features that explains the greatest proportion of the variance** in our data
- Reduce the size of the model
  - Size = number of model's features
  - Size ≠ number of inputs ("variables")
- Ensure features are independent of each other
  - PCA guarantees that all features are uncorrelated
  - PCA uses a linear combination of inputs as "new" features
  - Each principal component is orthogonal to all others
- Side effect: lose model "interpretability"... but may gain insight
  - Graphing data to show clusters, separation boundaries
  - Deciding on a model to use

02 STEPS OF PCA

STEPS OF PCA

PCA: How, Why and When

- How
  - Eigendecomposition
  - Typical to standardize the data prior to application of PCA
- Why
  - PCA captures most of the variance in the data
  - Projection: linear transformation of correlated input variables into smaller number of uncorrelated variables
  - PCA is a projection to reduced space, e.g., a projection from 3D to 2D
- When
  - Collinearity exists (i.e. features are related to each other)
  - Data is noisy and noise needs to be removed or compressed
  - The data dimensions are high (i.e. lots of features) and needs to be reduced

STEPS OF PCA

A Note on Standardization vs. Normalization

- Why do we need data scaling with PCA?
  - PCA is based on variance, and features with larger scales will dominate without proper scaling
- Standardization
  - Centers data at 0 mean and standard deviation of 1
  - Subtract the mean from each data point and divide by the std
  - Best for PCA where variance matters
- Normalization
  - Scales values (i.e., constrains them) from 0 to 1 (with min-max scaling)
  - Not ideal for PCA since it distorts variance relationships

$$z = \frac{x - mean(x)}{std(x)}$$

STEPS OF PCA

Steps of PCA - Overview

1. Center the data (remove the mean → we cover this by standardizing our data)
2. Generate a covariance matrix
3. Extract the eigenvectors (i.e., components, *direction* of most variance possible for each orthogonal dimension) and eigenvalues (i.e., *how much variance* captured) from the covariance matrix
4. Rank the eigenvalues for the scree plot
5. Select the number of components
6. Get loadings of each variable corresponding to each component (with eigenvector multiplied by $\sqrt{\text{eigenvalue}}$) and project data on the selected number of components

This transforms the data into a lower-dimensional space

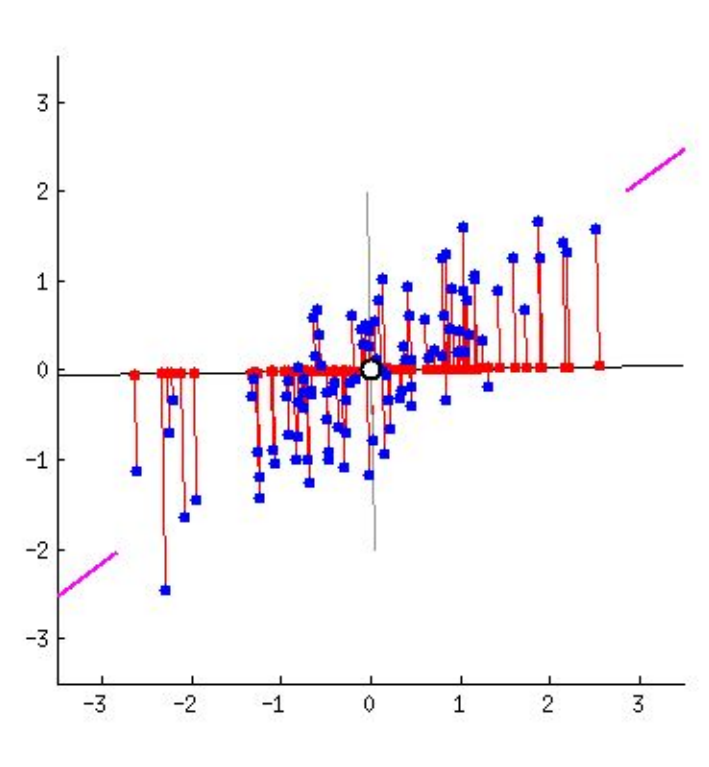
STEPS OF PCA

Finding Principal Components

Imagine two variables (one on x-axis and one on y-axis)

We want to project our data onto a 1D space

What are the “goals” of PCA when finding the principal component?



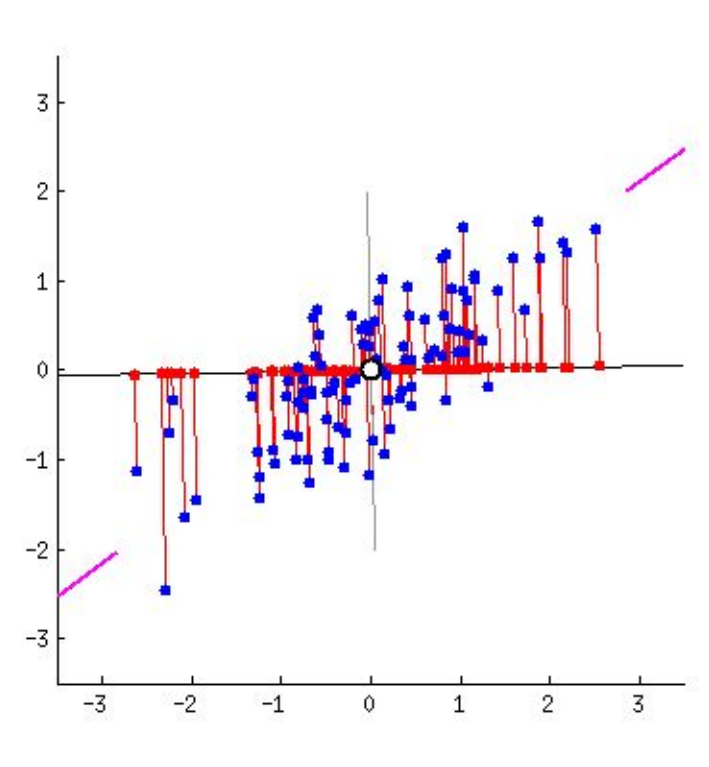
STEPS OF PCA

Finding Principal Components

Want to find the “best” line where:

1. Variation is maximal
2. Error is minimal

What position is variation (spread) of the projected data the biggest? Smallest?



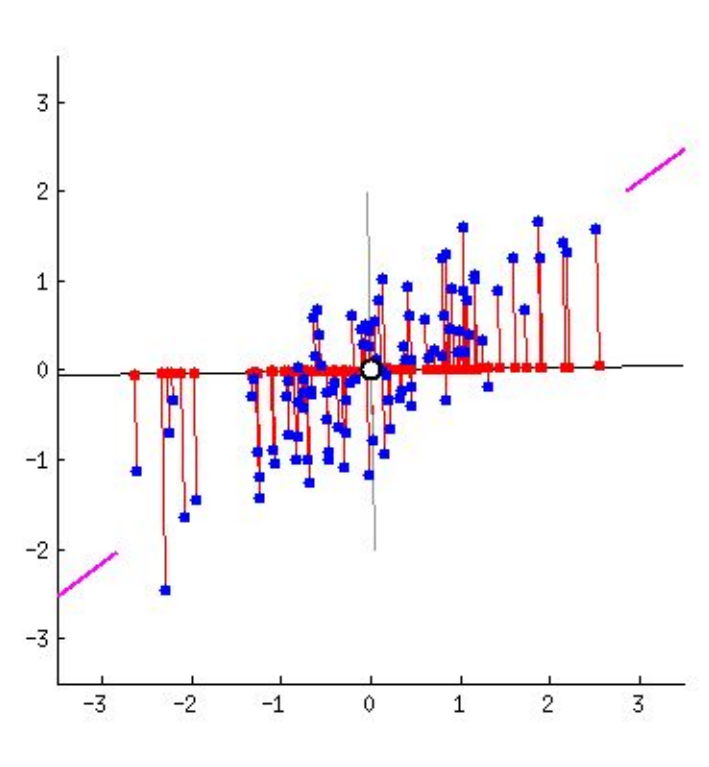
STEPS OF PCA

Finding Principal Components

Want to find the “best” line where:

1. Variation is maximal
2. Error is minimal

What position is error (distance from data point to line) smallest? Biggest?



STEPS OF PCA

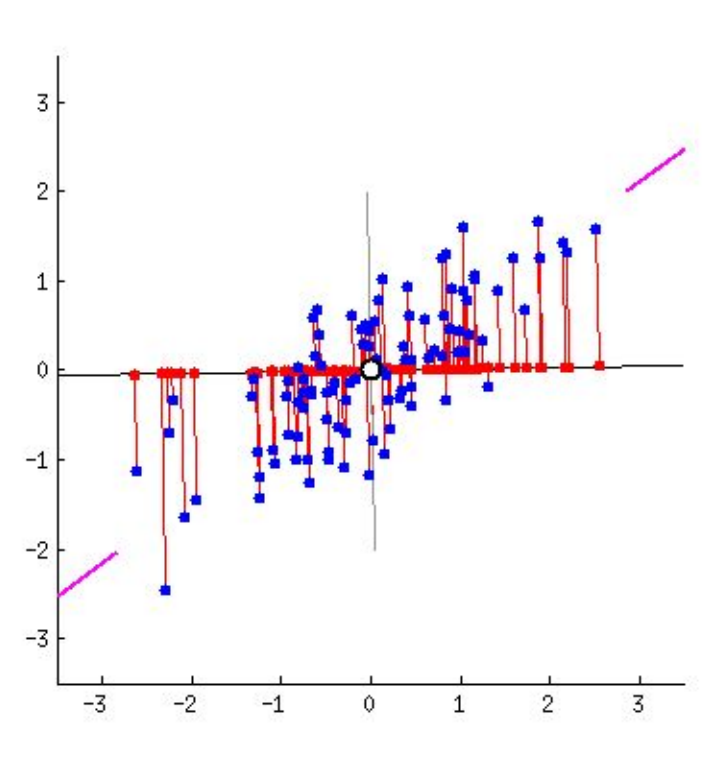
Finding Principal Components

Want to find the “best” line where:

1. Variation is maximal
2. Error is minimal

When this line is in alignment with the magenta ticks we are set!

This position is our PC1 → the component that captures the *most* variance and has the *least* error in terms of the projected data (red dots)



STEPS OF PCA

Steps of PCA - Overview

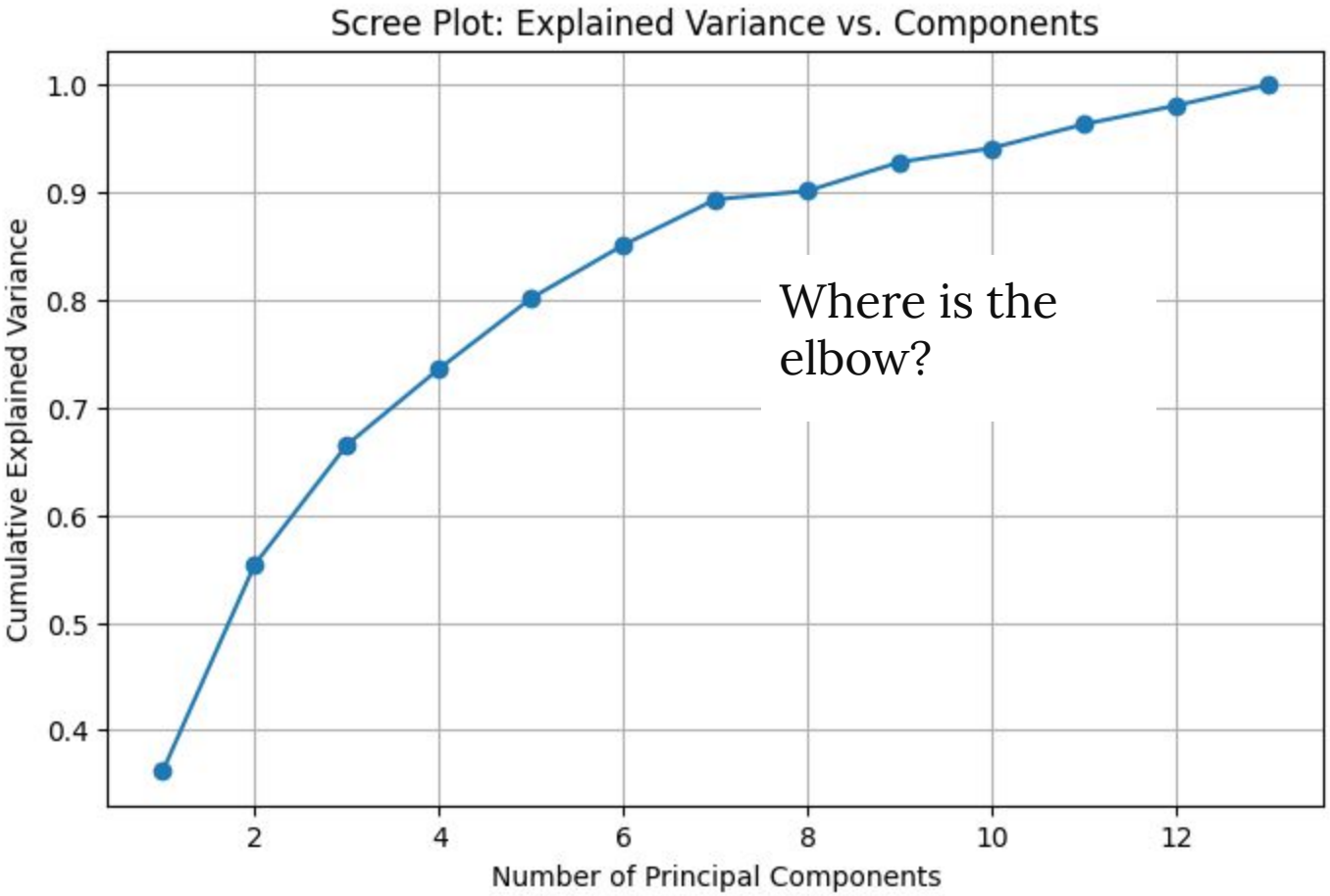
1. Center the data (remove the mean → we cover this by standardizing our data)
2. Generate a covariance matrix
3. Extract the eigenvectors (i.e., components, *direction* of most variance possible for each orthogonal dimension) and eigenvalues (i.e., *how much variance* captured) from the covariance matrix
4. **Rank the eigenvalues for the scree plot**
5. **Select the number of components**
6. Get loadings of each variable corresponding to each component (with eigenvector multiplied by √eigenvalue) and project data on the selected number of components

This transforms the data into a lower-dimensional space

Scree Plot for Selecting Components

Tells us the cumulative proportion of variance explained as we add principal components

How much explained variance are we aiming for typically?



STEPS OF PCA

Steps of PCA - Overview

1. Center the data (remove the mean → we cover this by standardizing our data)
2. Generate a covariance matrix
3. Extract the eigenvectors (i.e., components, *direction* of most variance possible for each orthogonal dimension) and eigenvalues (i.e., *how much variance* captured) from the covariance matrix
4. Rank the eigenvalues for the scree plot
5. Select the number of components
6. **Get loadings of each variable corresponding to each component** (with eigenvector multiplied by √eigenvalue) and project data on the selected number of components

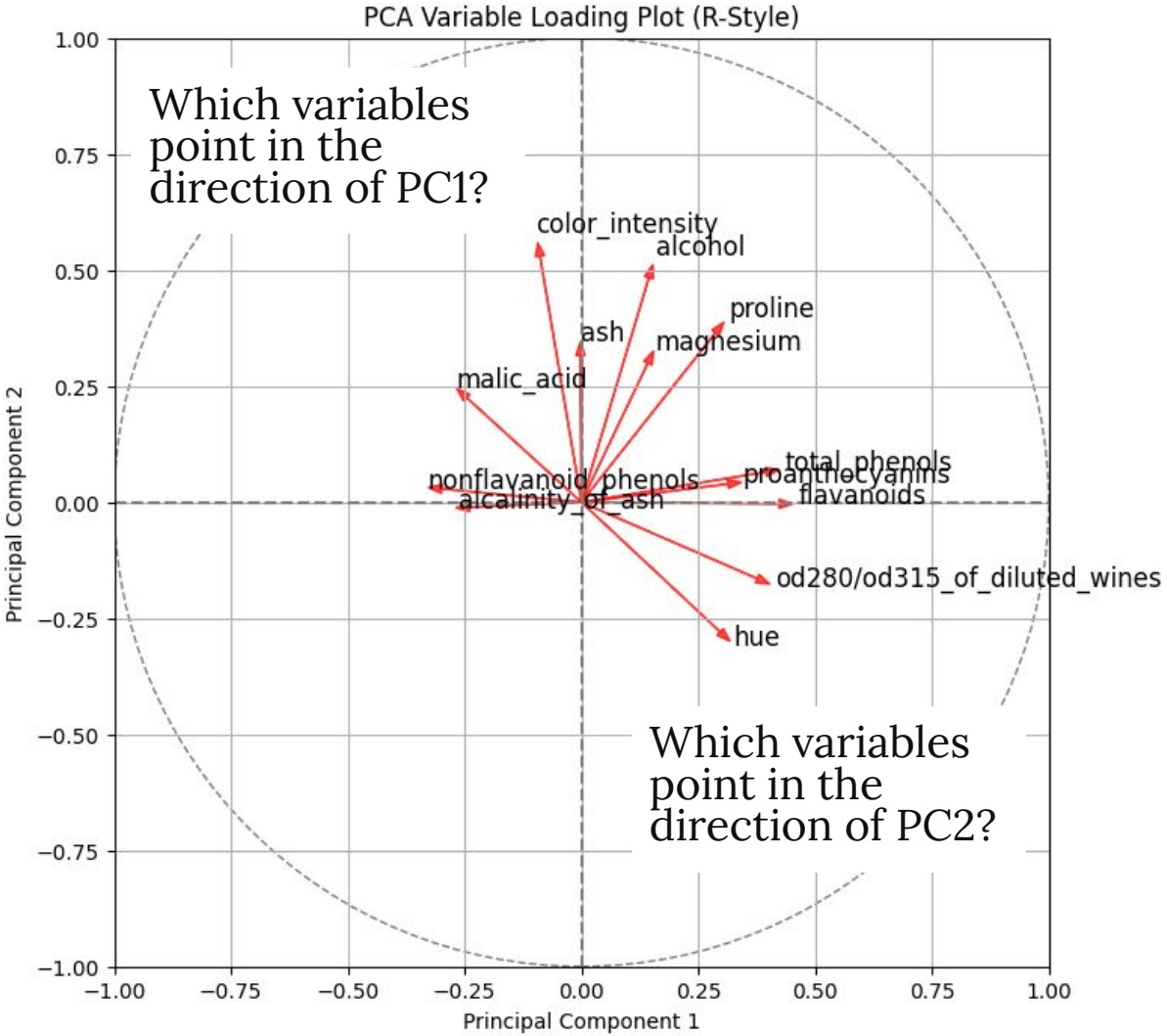
This transforms the data into a lower-dimensional space

Plotting Variables in Relationship to PCs

Biplots use the component loadings to give us something interpretable back...

Which of our variables influence different principal components

Looking at PC1 and PC2 is very valuable because this is usual a very large proportion of the variance explained



STEPS OF PCA

Steps of PCA - Overview

1. Center the data (remove the mean → we cover this by standardizing our data)
2. Generate a covariance matrix
3. Extract the eigenvectors (i.e., components, *direction* of most variance possible for each orthogonal dimension) and eigenvalues (i.e., *how much variance* captured) from the covariance matrix
4. Rank the eigenvalues for the scree plot
5. Select the number of components
6. Get loadings of each variable corresponding to each component (with eigenvector multiplied by $\sqrt{\text{eigenvalue}}$) and project data on the selected number of components

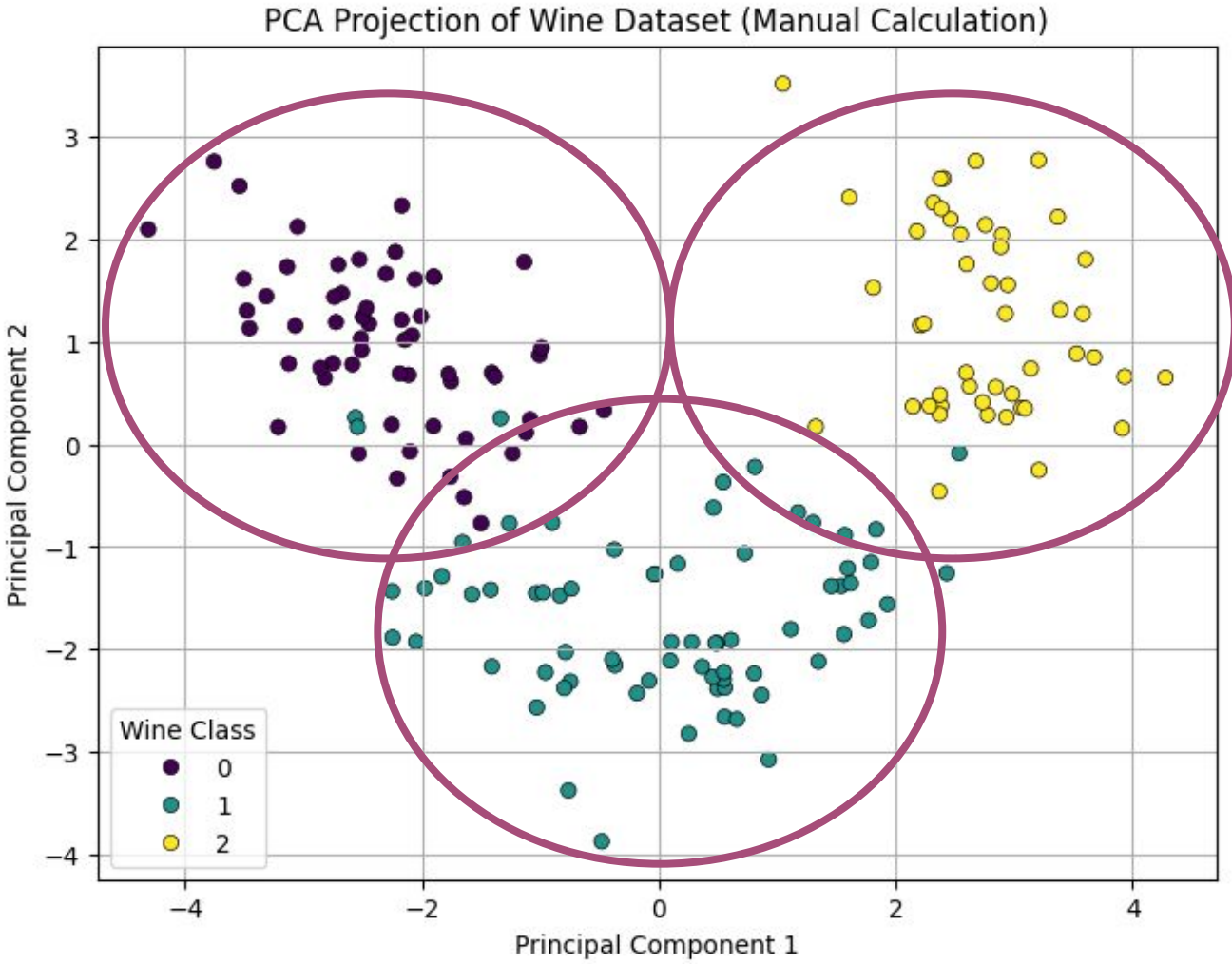
This transforms the data into a lower-dimensional space

Data Projection with Only 2 Components

The projection of the data onto the first two components

What can we infer about the proportion of

Not all data will cluster this nicely



03 MATH OVERVIEW OF PCA

Steps in Python

- Center the data (i.e., subtract away the mean)

```
# this is the df transposed where each nested list is a single variable
toy_wine = [[12, 13, 12, 14, 14],[5, 2, 3, 2, 3],[0, 1, 1, 2, 1]]

# Average each sublist (get the sample mean of each variable)
sum_vars = np.matrix([[65], [15], [5]])
avg_vars = 1/5 * sum_vars
print(avg_vars)

# Convert toy_wine to a NumPy array for easier manipulation
toy_wine_np = np.array(toy_wine)

# Subtract avg_vars from each column
# This is like standardizing the data (centering the mean)
centered_data = toy_wine_np - avg_vars
```

MATH OVERVIEW OF PCA

Steps in Python

- Generate a covariance matrix with the centered data

```
# Result back as a list of lists:
A = centered_data.tolist()
A = np.matrix(A)

# dot product (multiply row by column entries and sum)
# row_number by column_number final entry value in row_number, column_number position
# 2nd row by 3rd column final value in (2, 3) position in final matrix
C = A*(A.transpose())

C = 1/5*C
# Covariance matrix
print(C)
```

MATH OVERVIEW OF PCA

Steps in Python

- Extract eigenvectors and eigenvalues from covariance matrix

```
# Need to find the eigenvectors (the component loadings i.e., the contribution of
# each variable to a particular component)
eigenvalues, eigenvectors = np.linalg.eig(C)
print(eigenvalues)
print(eigenvectors)

# Compute the loadings (eigenvectors scaled by sqrt of eigenvalues)
loadings = eigenvectors * np.sqrt(np.diag(eigenvalues))

# Project the centered data onto the principal components
projected_data = eigenvectors.T * A
```

Steps in Python

- Extract the loadings (for visualization/interpretability) and project the data combining the eigenvectors and centered data

```
# Compute the loadings (eigenvectors scaled by sqrt of eigenvalues)
loadings = eigenvectors * np.sqrt(np.diag(eigenvalues))

# Project the centered data onto the principal components
projected_data = eigenvectors.T * A
```

SUMMARY

Summary

- PCA allows us to describe a dataset
  - With fewer than dimensions than our original dataset (some loss of resolution)
  - Lower-dimensional dataset may be useful for model speed & accuracy
  - Each principal component is orthogonal to the others
- Variance
  - PC1 explains the maximum variance in the data
  - All subsequent components explains less variance than the previous
  - A scree plot shows variance
- Practice
  - PCA is sensitive to outliers → standardize data prior to use
  - Use sklearn.decomposition.PCA